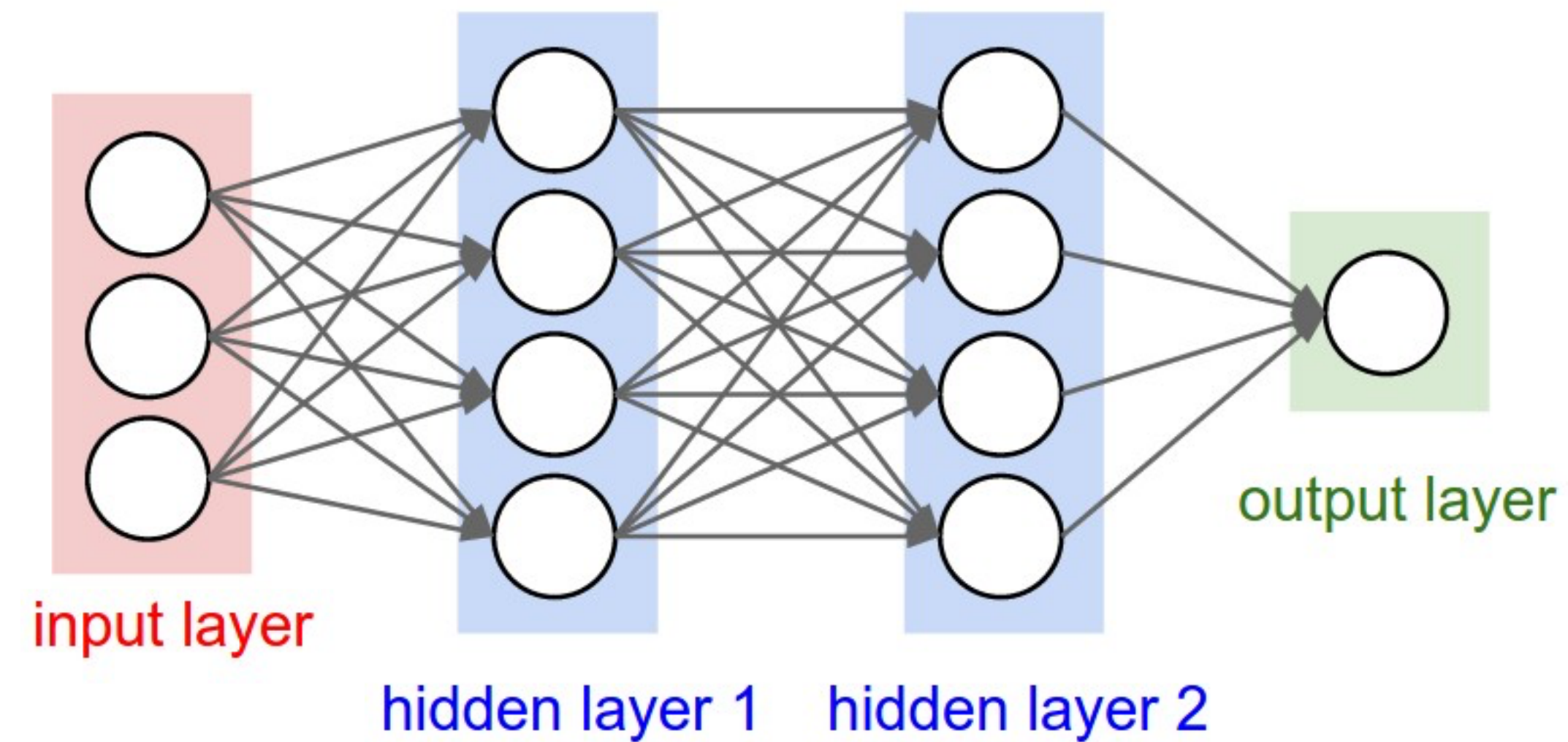


Backpropagation

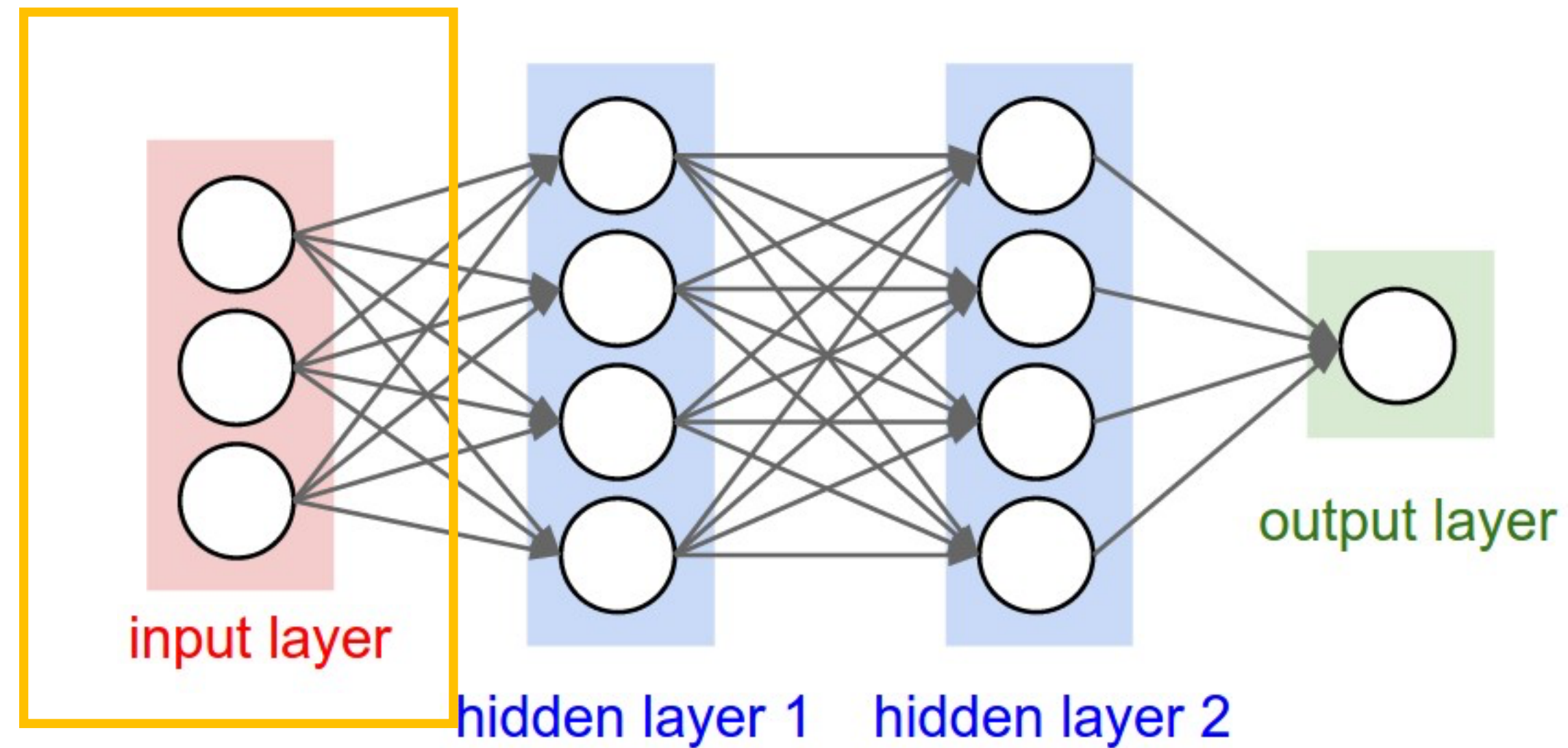
CSC 487: Deep Learning

Jonathan Ventura

Neural network represents a non-linear function

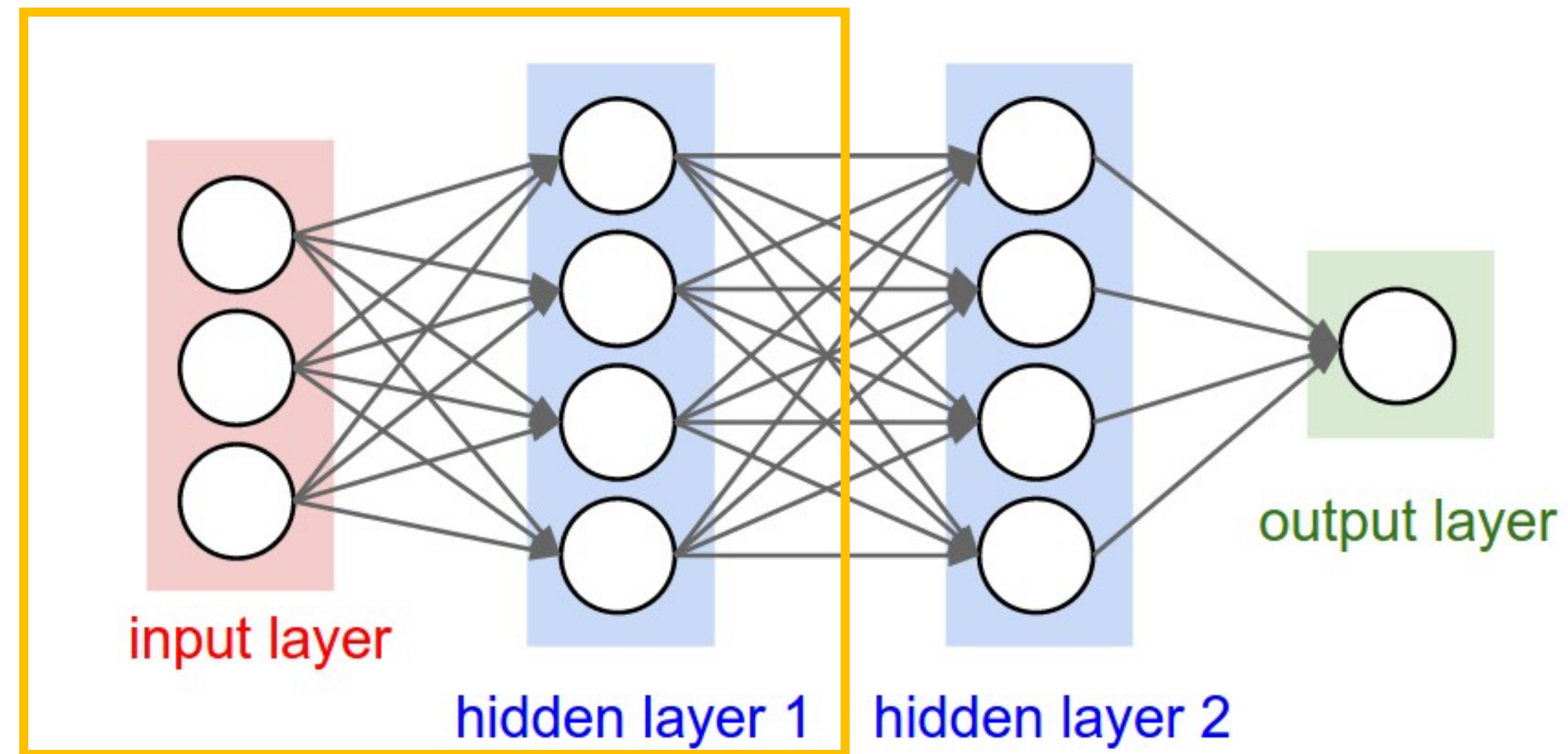


Neural network represents a non-linear function



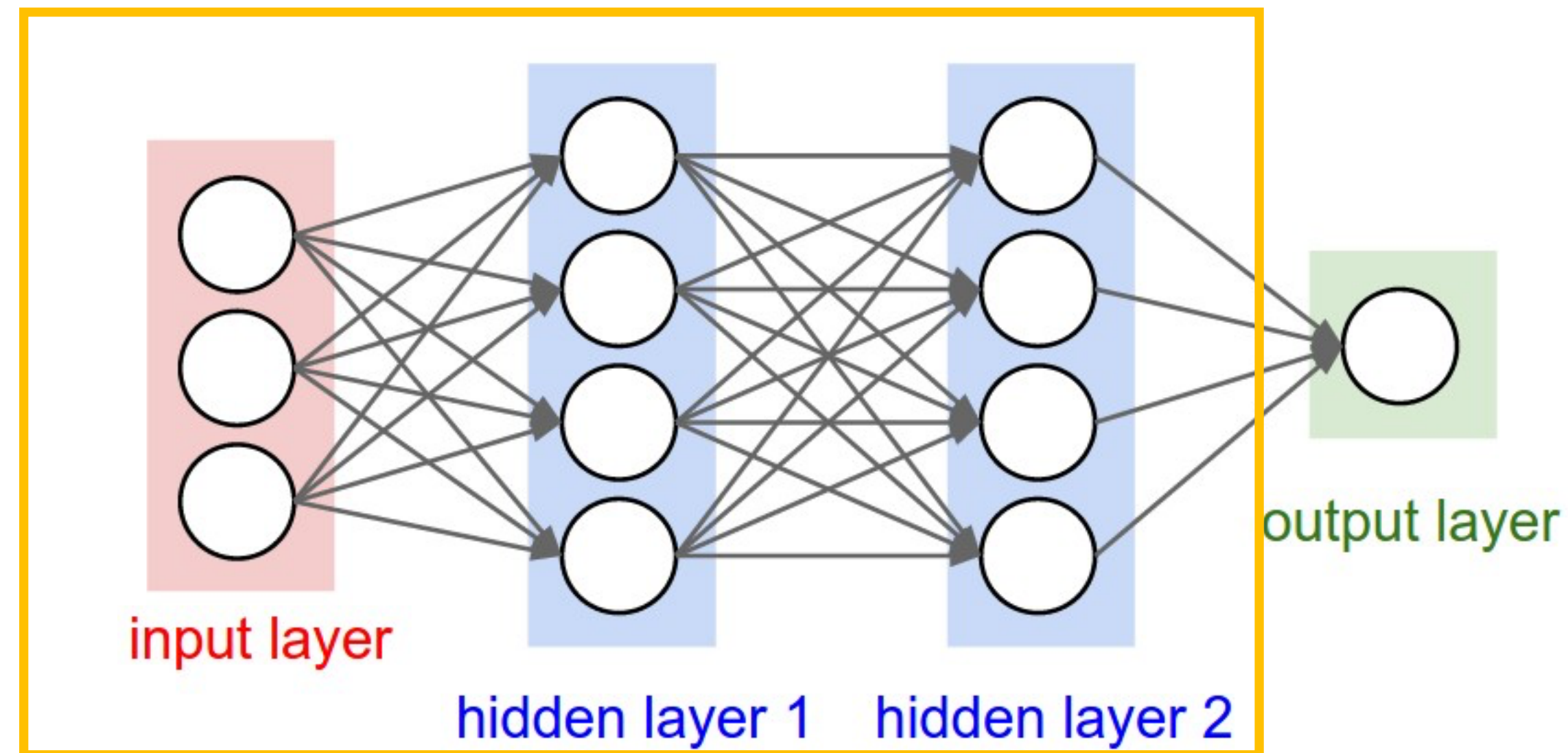
x

Neural network represents a non-linear function



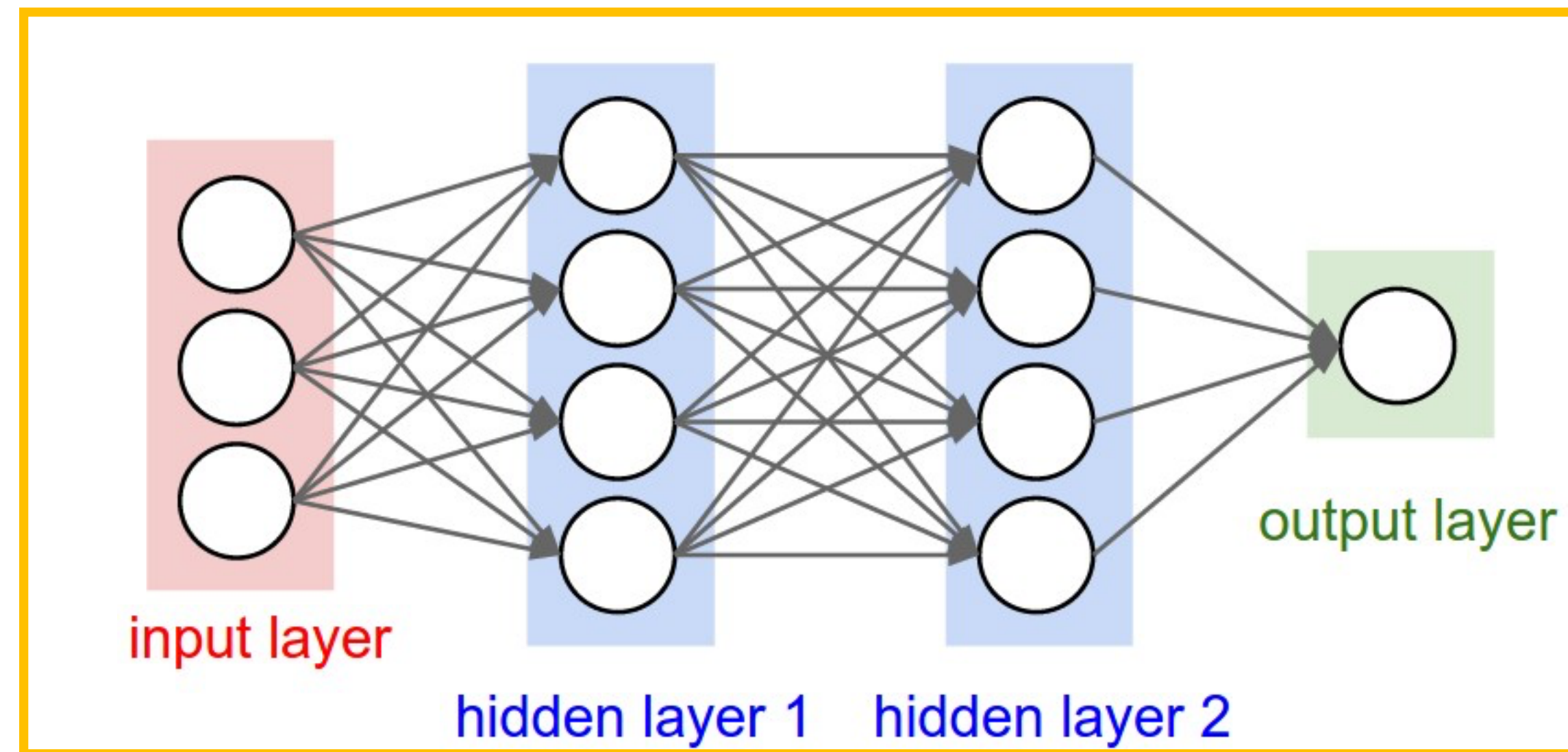
$$\sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)$$

Neural network represents a non-linear function



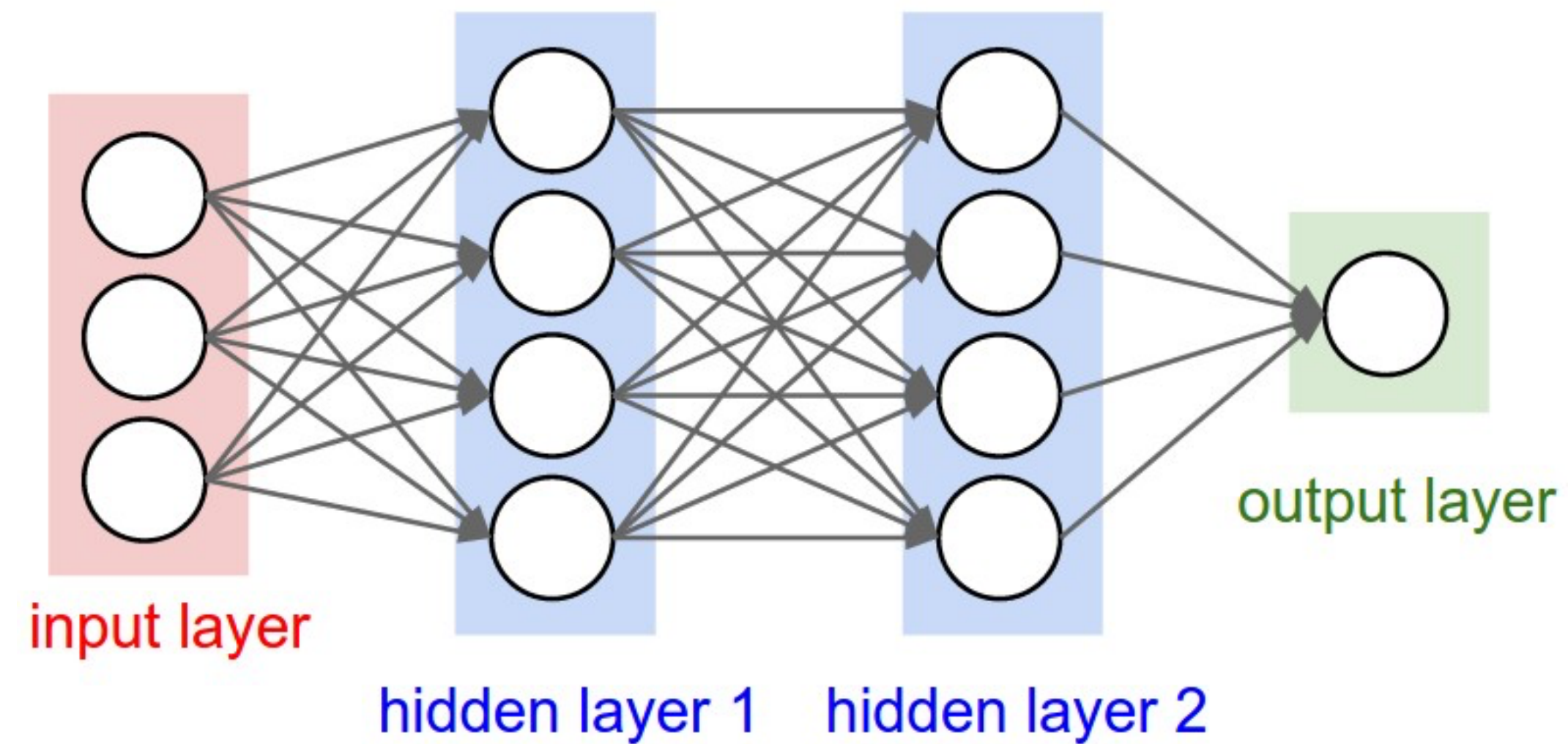
$$\sigma(\mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2)$$

Neural network represents a non-linear function



$$\mathbf{W}_3\sigma(\mathbf{W}_2\sigma(\mathbf{W}_1x + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3$$

Adding a loss function



$$L = \frac{1}{2}(\mathbf{W}_3\sigma(\mathbf{W}_2\sigma(\mathbf{W}_1x + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3) - y)^2$$

Training the network

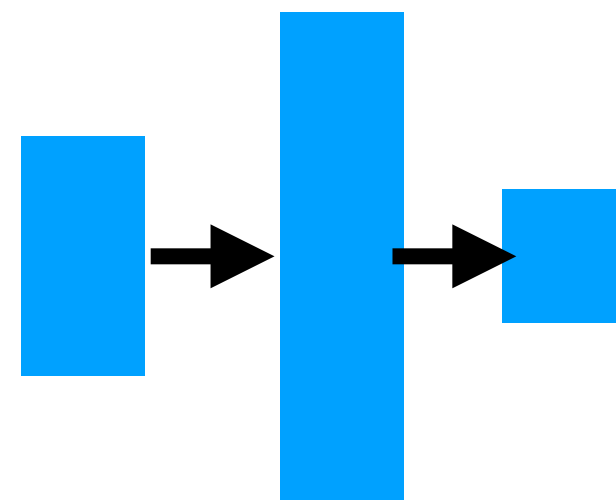
- Gradient descent:

$$w_j = w_j - s \cdot \frac{\partial L}{\partial w_j}$$

- How do we calculate $\frac{\partial L}{\partial w_j}$?

Single hidden layer example

- Assume in the remaining that we have “hidden” the bias into the weights.



$$\mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x}) = \mathbf{z}(\sigma(\mathbf{h}(\mathbf{x})))$$

$$\mathbf{h}(\mathbf{x}) = \mathbf{W}_1 \mathbf{x}$$

$$\mathbf{z}(\mathbf{x}) = \mathbf{W}_2 \mathbf{x}$$

Single hidden layer example

$$\frac{\partial}{\partial \mathbf{w}_1} L(\mathbf{z}(\sigma(\mathbf{h}(\mathbf{x})))) = ?$$

Remember the chain rule:

$$(f(g(x)))' = f'(g(x))g'(x)$$

Generalized Chain Rule

- Suppose we have a function $L(f_1(x), f_2(x), \dots, f_n(x))$.
- How to apply chain rule to compute $\frac{\partial L}{\partial x}$?
- The *generalized chain rule* for multivariate functions:

$$\frac{\partial L}{\partial x}(x) = \sum_{i=1}^n \frac{\partial L}{\partial f_i} \frac{\partial f_i}{\partial x}$$

Generalized Chain Rule

- Suppose we have a function $L(\mathbf{z}(w_j))$.
- How to apply chain rule to compute $\frac{\partial L}{\partial w_j}$?
- The *generalized chain rule* for multivariate functions:

$$\frac{\partial L}{\partial w_j}(w_j) = \frac{\partial L}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial w_j}$$

$1 \times |\mathbf{z}| \quad |\mathbf{z}| \times 1$

Generalized Chain Rule

- Suppose we have a function $L(\mathbf{z}(\mathbf{w}))$.
- How to apply chain rule to compute $\frac{\partial L}{\partial \mathbf{w}}$?
- The *generalized chain rule* for multivariate functions:

$$\frac{\partial L}{\partial \mathbf{w}}(\mathbf{w}) = \frac{\partial L}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{w}}$$

$$1 \times |\mathbf{z}| \quad |\mathbf{z}| \times |\mathbf{w}|$$

Single hidden layer example

- Application of chain rule for derivatives:

$$\frac{\partial L}{\partial \mathbf{W}_1} = \frac{\partial L}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \sigma} \frac{\partial \sigma}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{W}_1}$$

Single hidden layer example

- Application of chain rule for derivatives:

$$\frac{\partial L}{\partial \mathbf{W}_1}(\mathbf{z}(\sigma(\mathbf{h}(\mathbf{x})))) = \frac{\partial L}{\partial \mathbf{z}}(\mathbf{z}(\sigma(\mathbf{h}(\mathbf{x})))) \frac{\partial \mathbf{z}}{\partial \sigma}(\sigma(\mathbf{h}(\mathbf{x}))) \frac{\partial \sigma}{\partial \mathbf{h}}(\mathbf{h}(\mathbf{x})) \frac{\partial \mathbf{h}}{\partial \mathbf{W}_1}(\mathbf{x})$$

Single hidden layer example

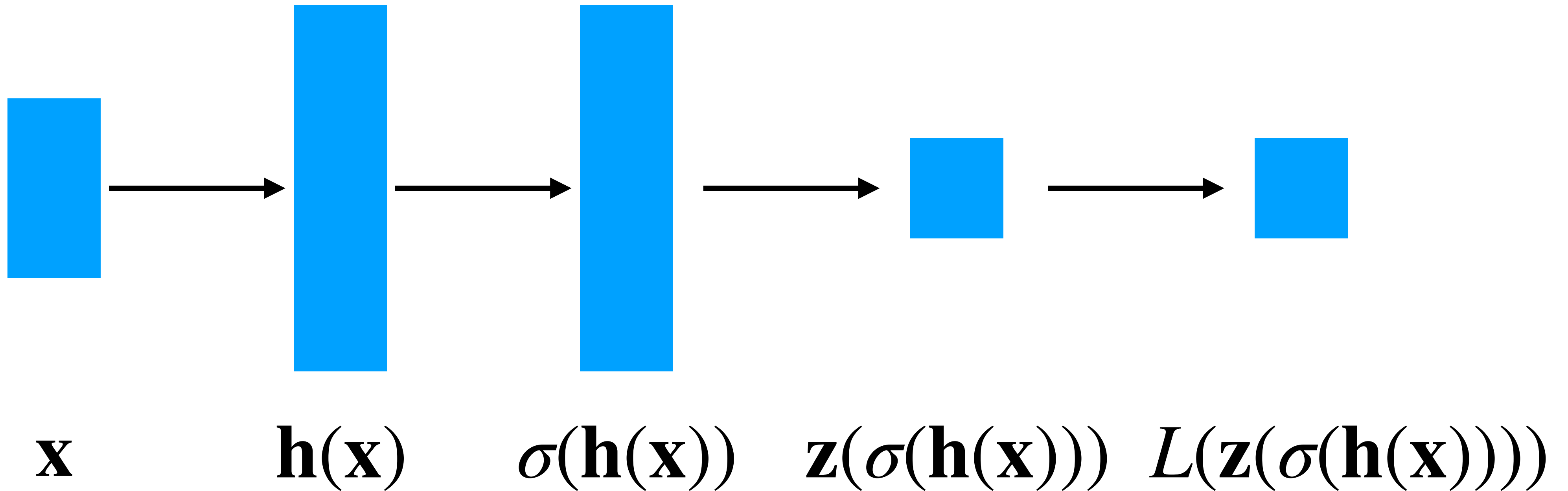
- How to compute this?

$$\frac{\partial L}{\partial \mathbf{W}_1}(\mathbf{z}(\sigma(\mathbf{h}(\mathbf{x})))) = \frac{\partial L}{\partial \mathbf{z}}(\mathbf{z}(\sigma(\mathbf{h}(\mathbf{x})))) \frac{\partial \mathbf{z}}{\partial \sigma}(\sigma(\mathbf{h}(\mathbf{x}))) \frac{\partial \sigma}{\partial \mathbf{h}}(\mathbf{h}(\mathbf{x})) \frac{\partial \mathbf{h}}{\partial \mathbf{W}_1}(\mathbf{x})$$

- Could evaluate each derivative from scratch
 - Lots of redundant computation — terms are reused each time

Forward pass

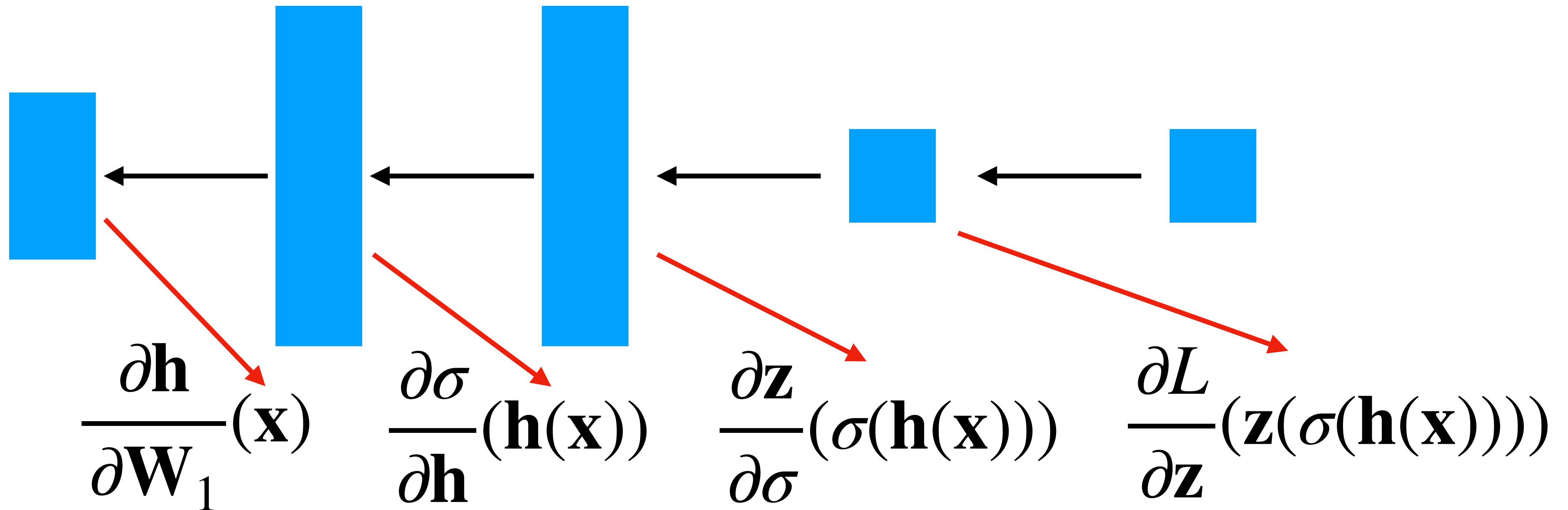
Propagate input forward through network



Save these intermediate values!

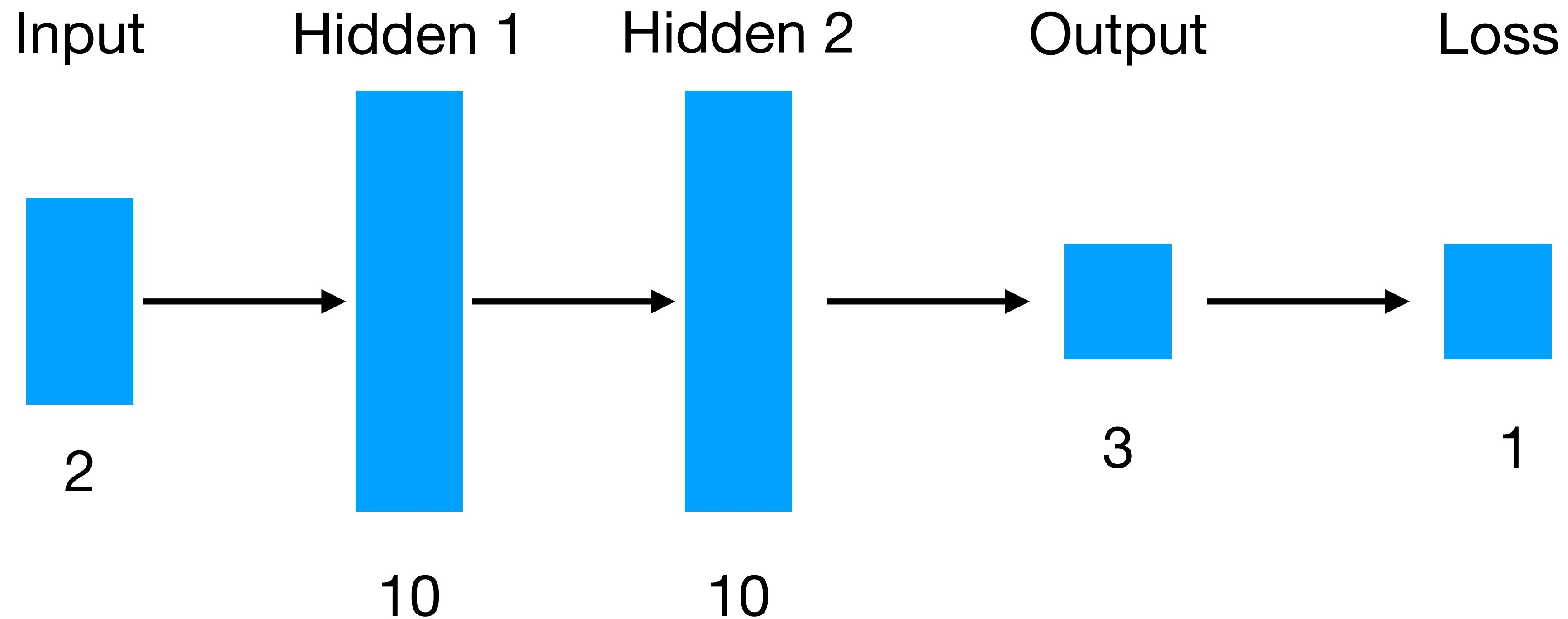
Backward pass

Propagate input forward through network



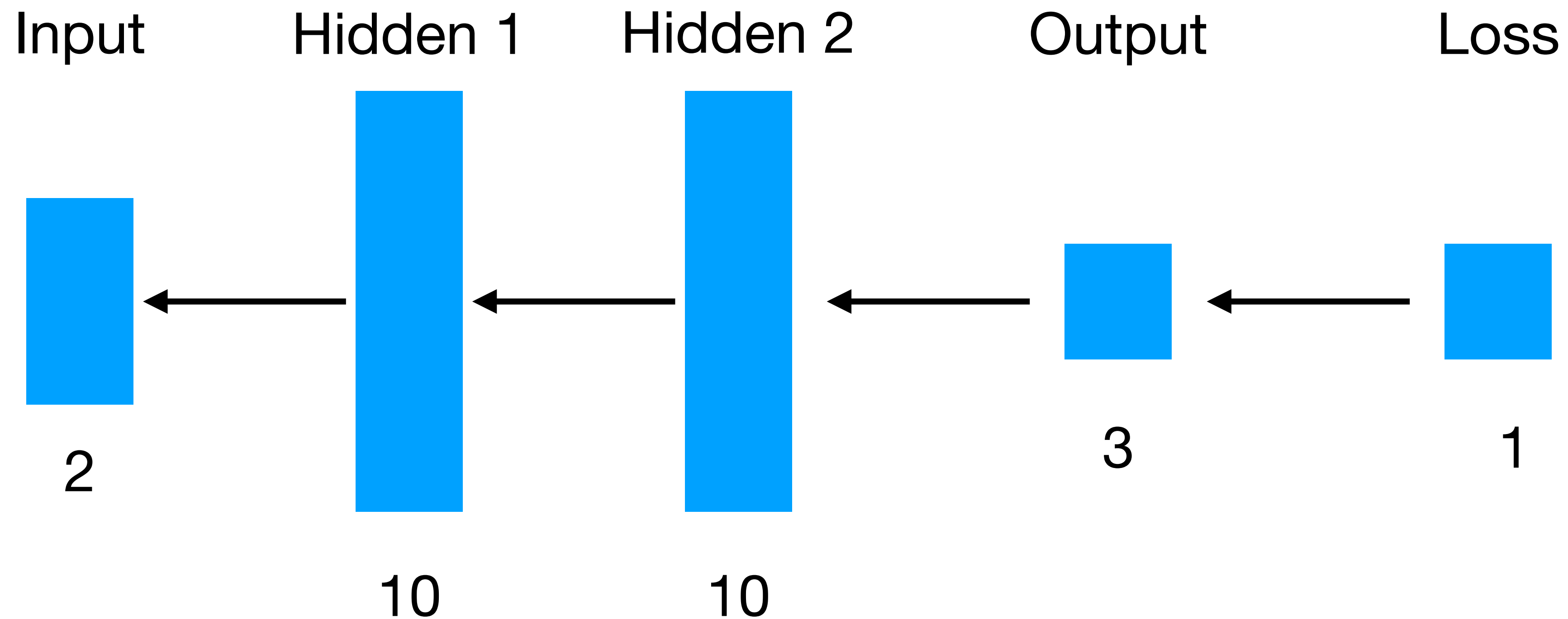
Reuse the intermediate values when computing derivatives
Update weights at each layer as error term propagates backward

Exercise



- Write down the sizes of all matrices used in the **forward** pass

Exercise



- Write down the sizes of all matrices used in the **backward** pass